



www.mech-lang.org

# A Rust-Native Embeddable Reactive Numerical Language for Heterogeneous Computing in Robotics

Corey Montella, Aung Kant Ko, Steven McPhillimey

## Motivation

- Rust’s memory safety and systems-level control are useful in robotics. Numerical kernels can still become lengthy and hard to check against the original equations, especially when each backend requires specialized code.
- Mech complements Rust by letting programmers express kernels as typed matrix equations. Its compiler handles backend specialization, and its reactive runtime supports successive sensor updates. We examine these features using a bearing-only extended Kalman filter (EKF).

## Embeddable

Rust applications can link Mech’s runtime to load source or bytecode, or use a compiled numerical kernel. The interface below compiles the EKF, binds live measurements and exposes persistent state by name. Rust functions and input drivers connect it to the surrounding application.

### AOT kernel from Rust

```
use mech::kernel::{Backend, Kernel};
let source = include_str!("ekf.mec");
let kernel = Kernel::from_source(source)
    .input("bearing", [-0.55; 4])
    .export("state")
    .compile(Backend::AotSimd)?;
let mut ekf = kernel.start()?;
ekf.turn([("bearing", [-0.54; 4])]);
let state = ekf.state("state");
```

Feature kernel-aot. compile emits the dylib, start creates a session, and turn applies one checked update to four filters. state borrows the accepted result. ekf.mec uses the f32 benchmark equations.

### Checked dylibs

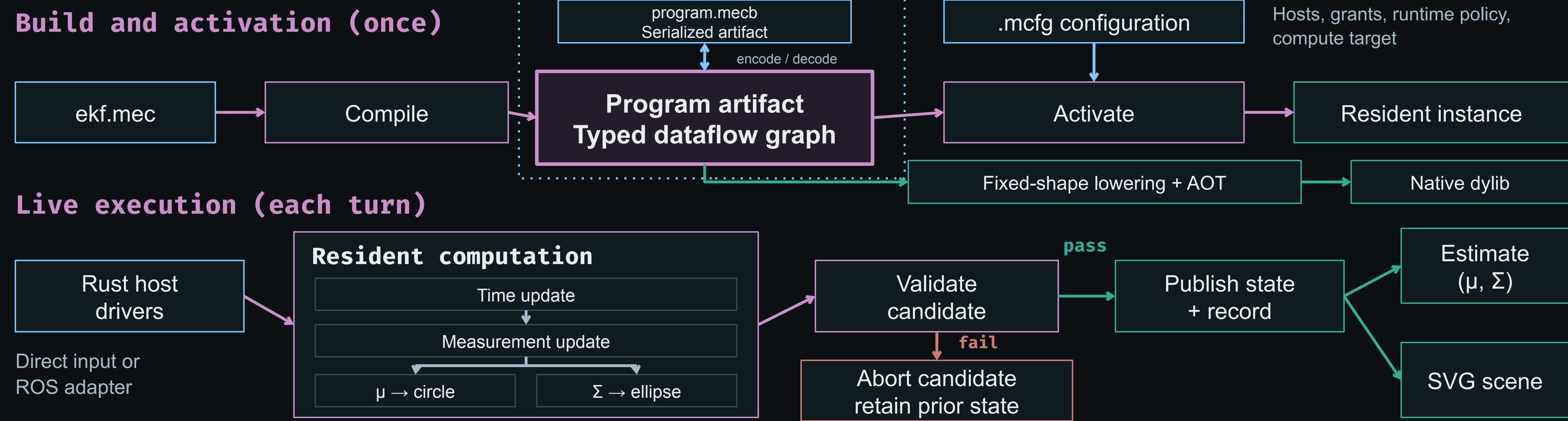
| Artifact    | Bytes  | M/s ± MAD      | Peak RSS (MiB) |
|-------------|--------|----------------|----------------|
| Mech scalar | 33,544 | 14.671 ± 0.008 | 2.69 ± 0.00    |
| Rust scalar | 50,016 | 21.121 ± 0.005 | 2.69 ± 0.00    |
| Mech SIMD-4 | 33,864 | 34.863 ± 0.010 | 2.69 ± 0.00    |

Mech’s smaller files primarily reflect linker and segment layout. Rust’s specialized scalar loop is faster than scalar Mech. Mech SIMD-4 packs four filters per vector and reaches higher throughput.

1 thread, 10,000 filters × 200 turns, n=7. Median ± MAD. RSS covers the whole loader process. Compile/load/warmup excluded.

## Reactive

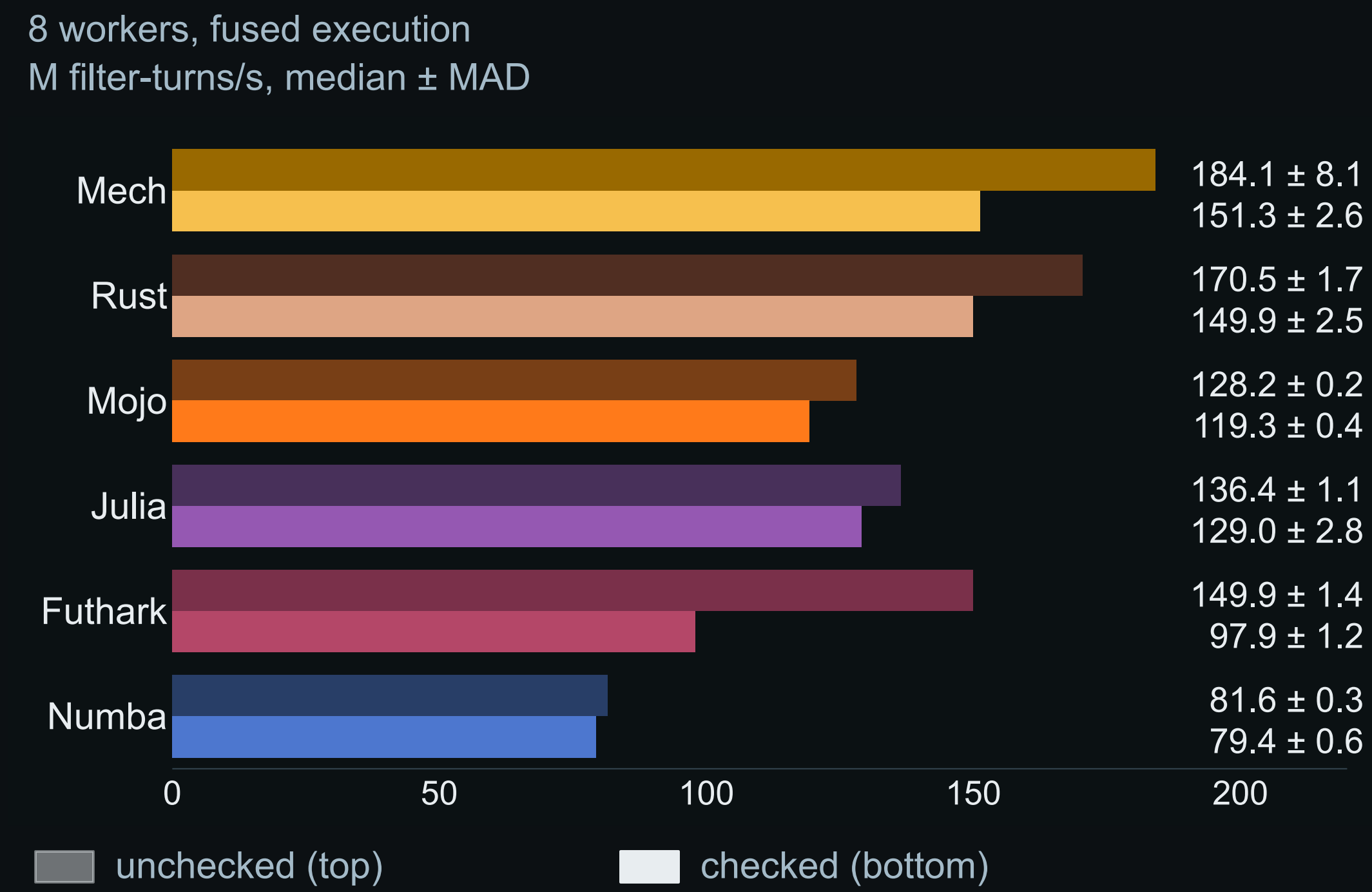
Robotics applications must incorporate new sensor data while retaining estimator state. Rust host drivers submit updates to the resident Mech graph, directly or through ROS adapters. A turn computes candidate state and validates it before publication. Rejection retains the previous state.



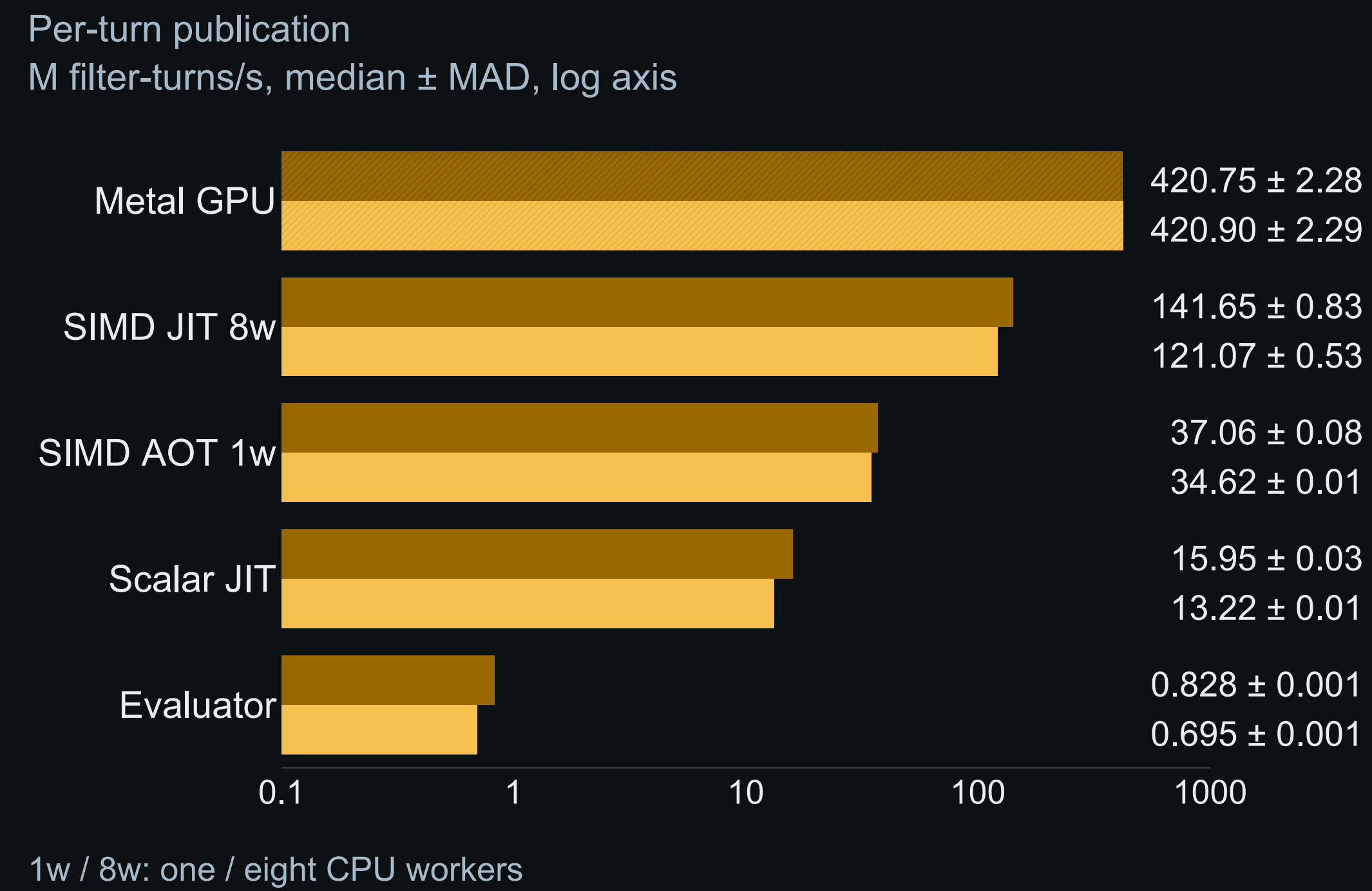
## Heterogeneous

All Mech backend measurements use one f32 source version of the EKF shown at right. SIMD JIT and AOT share the numerical graph and Cranelift code generator. JIT compiles on demand. AOT emits a native library before deployment, allowing the host to omit the compiler.

### Mech and Rust show similar throughput

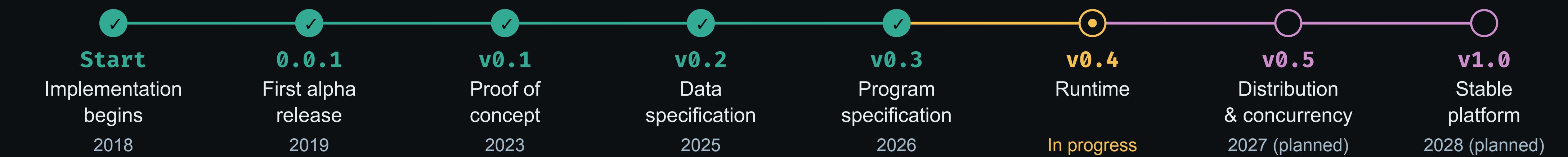


### Backend choice changes EKF throughput



## Status and Future Work

Mech is in beta. The EKF results show that shorter numerical code can achieve performance comparable to specialized Rust. Stable embedding APIs and tests across more numerical workloads are needed as the runtime and backend support develop.



Experience this poster as a live reactive program

Mech’s runtime compiles to WebAssembly for browser execution. Compare CPU and WGPU backends online.

## Numerical

Mech offers a concrete syntax that expresses numerical algorithms as typed matrix equations. The EKF below is comparable in scale to the textbook pseudocode [1], retaining its matrix-level notation.

SIMD Rust uses almost five times as many normalized source characters here, mainly for SIMD and batch support. Reusing the Mech source across backends reduces the specialized implementations to maintain.

Compilation produces a typed dataflow artifact. Matrix operations use nalgebra in the ordinary backend; specialized backends compile the graph directly.

Extended Kalman Filter

```
+> math/*
Δt := 0.1
m := [140 12]'
Q := [0.01 0
      0 0.0025]
R := 0.25
I := [1 0 0
      0 1 0
      0 0 1]
~μ := [55 25 0.4]'
~Σ := [100 0 0
      0 100 0
      0 0 0.15]
```

Time Update

```
θ := μ[3]
d := v * Δt
G := [1 0 (0 - d * sin(θ))
      0 1 d * cos(θ)
      0 0 1]
V := [cos(θ) * Δt 0
      sin(θ) * Δt 0
      0 Δt]
```

```
μ̄ := μ + [d * cos(θ)
          d * sin(θ)
          ω * Δt]
Σ̄ := G ** Σ ** G' + V ** Q ** V'
```

Measurement Update

```
δ := m - μ̄[1..=2]
δx := δ[1]
δy := δ[2]
q := δx ^ 2 + δy ^ 2

ẑ := atan2(δy, δx) - μ̄[3]
r := z - ẑ
v := atan2(sin(r), cos(r))

H := [δy / q, (0 - δx) / q, -1]
S := H ** Σ̄ ** H' + R
K := (Σ̄ ** H') / S[1]
A := I - K ** H

μ+ := μ̄ + K * v
Σ+ := A ** Σ̄ ** A' + (K ** K') * R

μ = μ+
Σ = Σ+
```

### Normalized source characters

| Application source     | Mech  | Rust textbook | Rust SIMD |
|------------------------|-------|---------------|-----------|
| Setup                  | 340   | 486           | 561       |
| EKF + candidate update | 356   | 1,024         | 1,119     |
| Validation             | 383   | 263           | 396       |
| Matrix helpers         | 0     | 251           | 310       |
| SIMD + batch support   | 0     | 331           | 2,857     |
| Total                  | 1,079 | 2,355         | 5,243     |

SIMD Rust uses almost five times as many normalized source characters as Mech in this example, mainly for SIMD and batch support.

Each identifier occurrence counts as one character. Comments and nonliteral whitespace are removed. General runtime/compiler internals are excluded.

### Benchmark setup

Apple M1: 4P + 4E CPU, 8-core GPU, 8 GB, macOS 15.6.1. Charts: f32, 500,000 filters × 40 turns, n=10. Median ± MAD.  
Rust 1.96.0-nightly ec818fda3 / LLVM 22.1.0, Cranelift 0.131.3. Julia 1.12.6. Mojo 1.1.0 / MAX 26.6.0. Futhark 0.27.1 / ISPC 1.31.0 / LLVM 22.1.8. Python 3.12.14 / NumPy 2.2.6 / Numba 0.61.2. Apple clang 17.0.0.  
Mech v0.4 development (package 0.3.6). Backend pairs f4b69052c, CPU eedc1c75b, dylibs dab98af1d. iros-workshop-benchmarks, benchmarks/iros-2026.

[1] Thrun, Burgard & Fox. Probabilistic Robotics. MIT Press, 2005. §7.4.2, Table 7.2 (adapted).

